Does Spec-Driven Development Reduce Defects? An Empirical Test of Industry Claims Across 119 Open-Source Repositories

Brenn Hill
Independent Researcher

Abstract

Context. Spec-driven development (SDD) tools claim that writing specifications before implementation reduces defects, prevents rework, and improves code quality. No vendor has published empirical evidence for these claims. **Method.** We test five hypotheses derived from SDD vendor claims against 88,052 pull requests across 119 open-source repositories. We score 25,209 specification artifacts on the same quality dimensions SDD tools prescribe, trace defects via the SZZ algorithm, and compare each developer's spec'd PRs to their own unspec'd PRs. Twelve robustness checks include propensity score matching, complexity stratification, and AI/human subgroup analysis. **Results.** None of the five hypotheses are supported. Spec'd PRs introduce *more* defects, not fewer—developers spec their hardest work, and hard work produces defects (confounding by indication). Specification quality does not predict fewer defects ($p = 0.164$) or less rework ($p = 0.860$). Specifications do not constrain AI-generated code scope ($p = 0.997$). One exception: in repositories with no AI adoption, specifications are associated with less rework ($p = 0.014$)—the only protective signal, appearing where AI is absent. **Conclusion.** Specification artifacts proxy for task complexity, not quality improvement. The one protective signal appears in human-only workflows, not AI-assisted ones.

2012 ACM Subject Classification Software and its engineering → Software creation and management

Keywords and phrases spec-driven development, null result, defect prediction, SZZ, AI-assisted development, within-author fixed effects

Digital Object Identifier 10.4230/LIPIcs...

Acknowledgements The dataset, analysis scripts, and replication package are openly available on Zenodo at `https://doi.org/10.5281/zenodo.19415187` under a CC-BY 4.0 license.

1 Introduction

The rise of AI-assisted software development has produced a new category of developer tooling: specification-driven development (SDD) frameworks. These tools—including GitHub's Spec Kit [8] and Amazon's Kiro [5]—claim that writing formal specifications before implementation leads to fewer defects, less rework, and higher-quality code, particularly when AI agents perform the implementation.

Böckeler [6] defines spec-driven development as "writing a 'spec' before writing code with AI" where "[t]he spec becomes the source of truth for the human and the AI." The GitHub Copilot Academy frames it more strongly: "Specifications don't serve code—code serves specifications" [1]. The movement gained momentum in 2025 with the release of GitHub's Spec Kit (open-source) and Amazon's Kiro IDE. Böckeler examined three tools pursuing the approach [6]. More broadly, SDD is positioned as an antidote to "vibe coding"—iteratively prompting AI without formal requirements—which practitioners associate with scope creep, rework, and defects.

The core premise is intuitive: if developers articulate what software should do before writing it, AI systems will produce more reliable output. The GitHub Copilot Academy

© Brenn Hill;
licensed under Creative Commons License CC-BY 4.0

lists "reduced rework, living documentation, systematic quality" among SDD's benefits [1]. Kiro claims that specifications enable "fewer iterations and higher accuracy" and "prevent costly rewrites" [22]. GitHub's Spec Kit blog post promises "less guesswork, fewer surprises, and higher-quality code" [8].

These claims rest on a plausible causal chain: clearer requirements → less ambiguity → fewer defects. Yet vendors cite no empirical evidence for these claims in their marketing materials, documentation, or blog posts. No study has tested SDD's quality claims on production-scale data prior to this paper.

This paper provides the first large-scale empirical test. We analyze 88,052 merged pull requests across 119 open-source repositories, after excluding 12,195 bot-authored PRs. Defects are traced to their introducing commits using the SZZ algorithm (named for Śliwerski, Zimmermann, and Zeller), which works backward from bug-fixing commits through version control history to identify which earlier commits introduced each bug. We compare each developer's spec'd PRs to their own unspec'd PRs to control for developer-level confounding, and derive five hypotheses directly from vendor claims.

2 Related Work

2.1 Requirements Engineering and Defect Prevention

The claim that upfront specification prevents defects has deep roots. Boehm's cost-of-change curve [4] argued that requirements defects are exponentially cheaper to fix early, but subsequent work challenged its universality: Shull et al. [21] found the cost multiplier depends on project context, and agile methods showed iterative refinement can outperform upfront specification [3]. Direct tests of whether requirements quality predicts defects have found weak or context-dependent effects: Mund et al. [15] found only weak evidence across two studies, and Femmer et al. [10] found that the relationship between requirements smells and downstream defects was context-dependent. Our study confirms this pattern at larger scale.

2.2 SDD Literature

The academic literature on SDD perpetuates the pattern of unsubstantiated claims. Piskala [18] claims "controlled studies showing error reductions of up to 50%," citing a Red Hat Developer blog post [16] and an InfoQ article [11] as evidence. Neither source contains any controlled study, any experiment, or any quantitative data. The "50%" figure traces through a citation chain that ultimately rests on practitioner opinion pieces with no empirical backing. Guo et al. [12] claim SDD "guarantees software quality" without empirical validation. Marri [14] reports a 73% security defect reduction from constitutional specs, but on a single project ($n = 1$), with the same developer implementing both conditions sequentially—the author acknowledges "small sample size limits statistical power." Roy [19] proposes iterative spec-code co-evolution but reports no independently validated quality outcomes. Zietsman [24] runs small planted-bug experiments and is honest about their limitations ("directional, not statistically significant"), while contributing the useful observation that AI reviewers without specs suffer circular reasoning—but does not test whether specs reduce defects at scale.

Practitioner evaluations have been more skeptical. Eberhardt, writing for Scott Logic, found Spec Kit roughly ten times slower than iterative development, producing thousands of lines of specification markdown that still resulted in buggy code [9]. Zaninotto argued that "SDD shines when starting a new project from scratch, but as the application grows,

the specs miss the point more often and slow development," and proposed iterative natural-language development as a faster alternative [23].

3 Hypotheses

We identified five empirical claims made by SDD tool vendors in their public documentation, blog posts, and marketing materials. Each claim is stated verbatim and mapped to a testable hypothesis.

Claim 1: Specifications reduce defects.

GitHub's Spec Kit blog post promises "less guesswork, fewer surprises, and higher-quality code" [8]. Kiro claims specifications lead to "fewer iterations and higher accuracy" [22].

H1: Pull requests with specification artifacts have lower defect-introduction rates than pull requests without them, controlling for author and PR size.

Claim 2: Specifications reduce rework.

The GitHub Copilot Academy lists "reduced rework" among SDD's benefits [1]. Kiro claims to "prevent costly rewrites" [22]. GitHub's blog states that SDD enables "iterative development without expensive rewrites" [8].

H2: Pull requests with specification artifacts have lower rework rates than pull requests without them, controlling for author and PR size.

Claim 3: Clearer specifications produce more reliable code.

GitHub's Spec Kit blog states that "providing a clear specification up front, along with a technical plan and focused tasks, gives the coding agent more clarity, improving its overall efficacy" [8]. Kiro claims specifications enable "fewer iterations and higher accuracy" [22]. This implies a dose-response relationship: higher specification quality should predict fewer defects.

H3: Among pull requests with specifications, higher specification quality predicts lower defect-introduction rates, controlling for author and PR size.

H4: Among pull requests with specifications, higher specification quality predicts lower rework rates, controlling for author and PR size.

Claim 4: Specifications constrain AI-generated code scope.

GitHub's Spec Kit blog describes tasks that give "the coding agent a way to validate its work and stay on track" [8]. Kiro claims specs provide "a North Star to guide the work of the agent, allowing it to take on larger tasks without getting lost" [5].

H5: Specifications reduce code churn (additions + deletions) for AI-tagged pull requests more than for human-authored pull requests, controlling for author.

4 Methods

4.1 Dataset

We collected pull request data from 119 open-source repositories spanning 17 programming languages. Repositories were selected to represent large, active projects with diverse contribution patterns. We collected data between March 28 and April 3, 2026, with a 365-day lookback window per repository. The resulting dataset spans April 2025 through early April 2026 (1st–99th percentile: April 8, 2025 to April 1, 2026), totaling 100,247 PRs. We excluded 12,195 bot-authored PRs (dependabot, renovate, and other automated accounts identified by username pattern and bot flags), leaving 88,052 PRs from human and AI-assisted authors.

Table 1 Study dataset overview.

Metric	Value	Notes
Repositories	119	convenience sample, 17 languages
PRs (post-bot exclusion)	88,052	12,195 bots removed
Median PRs per repo	517	range 9–4,803
Median spec rate (per repo)	26.2%	IQR 16.0–41.1%
Median AI tag rate (per repo)	1.2%	IQR 0.4–8.1%
SZZ-covered repos	103	16 had unreachable merge SHAs
SZZ bug-introducing PRs	9,754	12.5% of SZZ-covered PRs
Quality-scored PRs	25,209	28.6% of total; 17,973 enriched with issue body

For each pull request, we extracted: author, merge date, additions, deletions, files changed, linked issues, PR description, review comments, and co-author tags (used to identify AI-assisted contributions).

The sample spans 17 programming languages, with Python (31 repos, 26%), TypeScript (26, 22%), Go (17, 14%), and Rust (13, 11%) predominating. The median repository has a 26.2% specification rate (IQR 16.0–41.1%). AI tagging is present in most repositories: the median AI tag rate is 1.2% (IQR 0.4–8.1%). The 5,989 AI-tagged PRs provide sufficient statistical power for the primary AI analyses, though we recognize that the AI-specific results would benefit from a larger AI-tagged sample as agentic workflows become more prevalent.

4.2 Variables

4.2.1 Specification Presence (`specd`)

A pull request is classified as "specified" (hereafter "spec'd") if it meets any of the following criteria: (a) it links to a GitHub issue or external tracking ticket, (b) its description references a specification document, RFC, or design doc, or (c) its description contains structured requirements content (acceptance criteria, behavioral descriptions, scope boundaries). Classification is performed programmatically by parsing PR metadata: issue references (`#123`, `fixes #456`), cross-reference URLs, and description structure (presence of headings, checklists, or requirement-style language). 24,298 of 88,052 PRs (27.6%) are classified as specified.

This operationalization is deliberately generous, capturing any form of upfront specification artifact. Specification *presence* and *quality* are tested separately: H1/H2 test whether having any spec predicts fewer defects or less rework; H3/H4 test whether, among spec'd PRs, higher quality predicts better outcomes.

4.2.2 Specification Quality (`q_overall`)

For the subset of specified PRs with substantial descriptions (body length > 50 characters), we scored specification quality by prompting Claude Haiku (Anthropic) with the PR title and body and a structured rubric. The LLM scored each PR across seven dimensions: outcome clarity, error states, scope boundaries, acceptance criteria, data contracts, dependency context, and behavioral specificity. Each dimension is scored 0–100, and the overall quality score is the mean across dimensions.

These seven dimensions were chosen to align with the quality criteria recommended by SDD tools themselves. Spec Kit's template prescribes "core requirements" (outcome clarity), "acceptance criteria" (our acceptance criteria dimension), "edge cases" (error states), "data models" and "API contracts" (data contracts), and "user stories" (behavioral specificity) [8]. Kiro prescribes "clear requirements," "acceptance criteria," "system architecture," and "technology choices" [22]. Six of our seven dimensions have direct equivalents in Spec Kit's recommended structure; all seven are covered across both tools. This alignment is deliberate: we measure specification quality on the dimensions that SDD vendors themselves say matter.

We validated the LLM scoring against independent human ratings on a stratified random sample of 38 PRs (blind to LLM scores). Overall agreement was moderate (Spearman $\rho = 0.42$, $p = 0.01$; Pearson $r = 0.45$, $p < 0.01$). Per-dimension agreement varies: four dimensions show significant agreement (behavioral specificity $\rho = 0.45$, scope boundaries $\rho = 0.43$, dependency context $\rho = 0.40$, outcome clarity $\rho = 0.39$), while three show weak correlation (error states $\rho = 0.22$, data contracts $\rho = 0.19$, acceptance criteria $\rho = 0.17$). The LLM reliably detects whether specification content is *present* but cannot judge whether it is *correct* for the domain—a distinction between formal completeness and functional quality. Since even this completeness-oriented measure shows no defect benefit, the null finding is conservative.

25,209 PRs (28.6% of total) received quality scores across all 119 repositories. For PRs linked to GitHub issues, we fetched the issue body and scored the combined content. 17,973 scores include linked issue content; the remainder were scored on PR description alone. This subsample is non-random—only spec'd PRs with sufficient text were scored.

A further limitation: we cannot distinguish specifications written before implementation from descriptions written after. Linked GitHub issues are the strongest available proxy for pre-implementation intent; we test this subset separately in Section 5.6.12. Quality scores for issue-linked PRs (median 49, $N = 10{,}190$) differ only trivially from description-only PRs (median 47, $N = 12{,}128$; $p < 0.001$ but 2-point difference). The null result holds for both subsets.

Furthermore, many effective AI instructions are inherently underspecified by our rubric's standards—a prompt like "improve the error handling" scores low but may be sufficient for an AI agent that can read surrounding code. This class of task is invisible to our quality measure but may represent a large share of productive AI-assisted work.

4.2.3 Defect Introduction (`szz_buggy`)

We applied the basic SZZ algorithm [25] to 103 of 119 repositories, tracing 64,805 blame links from fix commits to bug-introducing commits. The remaining 16 repositories had unreachable merge SHAs (typically from squash-merge workflows). da Costa et al. [7] found that basic SZZ misattributes 46–71% of bug-introducing changes. This noise is non-differential with respect to specification presence, attenuating associations toward the null without creating spurious reversals. A PR is marked `szz_buggy = True` if any of its commits were identified as bug-introducing. 9,754 PRs (12.5% of SZZ-covered repos) are marked as bug-introducing.

4.2.4 Rework (`reworked`)

A pull request is classified as reworked if a subsequent PR within 30 days by any author modifies overlapping files and has a title or description indicating correction (matching patterns such as "fix," "revert," "bugfix," "hotfix," "regression," "broke," or "broken"). File

overlap is computed by comparing the set of files changed in each PR pair; high-frequency files (touched by >30% of PRs in the repository) are excluded to avoid spurious attribution. 11,820 PRs (13.4%) are classified as reworked. The 30-day window was chosen based on sensitivity analysis: rework detection increases from 14 to 30 days but shows diminishing returns beyond 30 days (the spec–rework association is directionally consistent across 14-, 30-, 60-, and 90-day windows).

4.2.5 AI-Tagged (`ai_tagged`)

A pull request is classified as AI-assisted through two detection layers. First, a regex matches co-author tags (e.g., `Co-authored-by: Copilot`), generation attribution strings (`Generated with`, `Claude Code`, `written with/by` followed by a tool name), AI agent markers (`CURSOR_AGENT`, `coderabbit.ai`, `AI-generated`), and the robot emoji. Second, we parse structured "AI Disclosure" and "AI Usage" sections that some repositories include in their PR templates, classifying the disclosure content as AI-used or no-AI-used. We validated the combined classifier against 537 PRs with clear self-reported AI disclosures: precision is 98.8% (1 false positive) and recall is 24.2% (75.8% false negatives). The low recall reflects that many developers describe AI use in natural language that our patterns do not capture. 5,989 PRs (6.8%) carry AI tags after bot exclusion. This remains a lower bound.

4.2.6 Code Churn (`total_churn`)

Total code churn is defined as additions + deletions. We use $\log(1 + \text{churn})$ in regressions to normalize the heavily right-skewed distribution (median: 55, mean: 698).

4.3 Statistical Approach

For each hypothesis, we report three levels of analysis. Each level controls for more potential confounders than the last, so readers can see how the result changes as we account for more alternative explanations:

Pooled: Raw rates compared directly. This is the analysis that unadjusted claims would cite—it ignores all confounders.

Controlled: Logistic regression adjusting for PR size (log additions, log deletions, log files changed) and *repository fixed effects*, so the model compares spec'd and unspec'd PRs *within the same repo*. In two analyses (H1, H3), including 100+ repository adjustments caused singular matrices, so those estimates use size controls only.

Within-author: The strongest test. We subtract each developer's own average from every variable, then run a linear regression on the residuals. This compares each developer's spec'd PRs to their own unspec'd PRs, eliminating all stable differences between developers—skill, experience, coding style, and organizational context are held constant because both "treatment" and "control" come from the same person.[1] Standard errors account for the fact that PRs from the same author are correlated (clustered at the author level). Primary models use size controls; Section 5.6 confirms results hold when adding Just-In-Time (JIT) defect

[1] Technically: a linear probability model (LPM) with full author demeaning, equivalent to author fixed effects via the Frisch–Waugh–Lovell theorem. We use LPM rather than logistic regression because author demeaning is exact for OLS but produces biased estimates in logit due to the incidental parameters problem [17]. Chamberlain's conditional logit avoids this but requires discarding groups without outcome variation, reducing power. We follow Angrist and Pischke's recommendation [2].

Table 2 Sample sizes by hypothesis.

Hypothesis	PRs (pre-filter)	Within-author N	Identifying authors
H1: specs → bugs	77,814 (103 SZZ repos)	72,046	2,181
H2: specs → rework	88,052 (all 119 repos)	81,647	2,524
H3: quality → bugs	22,672 (scored, SZZ repos)	19,688	2,030
H4: quality → rework	25,209 (scored subset)	21,881	2,297
H5: spec × AI → churn	88,052 (all 119 repos)	71,932	366

Table 3 H1: Specifications and defect-introduction rates.

Method	Coefficient	p-value	Interpretation
Pooled (raw comparison)	OR = 1.20[a]	< 0.001	Spec'd PRs have *higher* defect rate
Controlled (size only[†])	0.067	0.006	Effect persists with size controls
Within-author	+0.014 [+0.005, +0.024]	0.003	+1.4pp *increase* in defect rate

prediction features [13]—a set of 14 code-change metrics (lines added, file age, developer experience, etc.) known to predict defect-introducing commits.

The within-author estimate is the most credible. The treatment effect is identified only from authors who have *both* specified and unspecified PRs in the dataset—authors who always or never write specs contribute no information.

We restrict within-author analysis to authors with ≥2 PRs and report the number of authors with treatment variation (those who have both spec'd and unspec'd PRs), as the treatment effect is identified from these authors' within-author variation. For within-author estimates, we report 95% confidence intervals based on clustered standard errors to allow readers to assess what effect sizes are compatible with the data.

We do not apply multiple-comparison corrections across the five hypotheses. At $\alpha = 0.05$, approximately 0.25 false positives are expected by chance. We interpret all results with this in mind.

Sample sizes vary across hypotheses because of SZZ coverage, quality scoring, and within-author filtering:

Within-author N is after restricting to authors with ≥2 PRs and dropping rows with missing controls. Identifying authors are those with variation in both treatment and control conditions.

5 Results

5.1 H1: Specifications Reduce Defects

Analysis restricted to 77,814 PRs in the 103 repositories with SZZ coverage.

[a] Odds ratio: spec'd PRs are 1.20× as likely to introduce a defect as unspec'd PRs. [†] Including a separate adjustment for each of 100+ repositories caused the model to fail; this estimate adjusts for PR size only. Within-author 95% CI in brackets.

Identified from 2,181 authors with treatment variation (out of 4,328 authors with ≥2 PRs).

H1 is not supported. Within-author, spec'd PRs are associated with a 1.4 percentage-

Table 4 H2: Specifications and rework rates.

Method	Coefficient	p-value	Interpretation
Pooled (raw comparison)	OR = 1.18[a]	< 0.001	Spec'd PRs have higher rework
Controlled (size + repo)	0.178	< 0.001	Effect persists
Within-author	+0.012 [+0.005, +0.019]	0.001	+1.2pp *increase* in rework rate

point *increase* in defect-introduction rates ($p = 0.003$). The effect is small[2] but the direction is opposite to the vendor claim at every level of analysis.

This pattern is *confounding by indication*: developers write specs for their hardest work, and harder work produces more defects. When JIT risk features are added as controls (Section 5.6), the spec coefficient drops 55% and loses significance ($p = 0.229$), confirming that task complexity explains most of the association. A detection bias alternative (spec'd PRs have more visible defects via SZZ tracing) is partially addressed by the rework measure (H2, not SZZ-dependent) showing the same pattern, and propensity score matching eliminating the association entirely.

5.2 H2: Specifications Reduce Rework

Analysis includes all 88,052 PRs.

Identified from 2,524 authors with treatment variation (out of 4,892 with $\geq$2 PRs).

H2 is not supported. The within-author effect is in the wrong direction: the same author's spec'd PRs have a 1.2 percentage-point higher rework rate than their unspec'd PRs ($p = 0.001$). The effect is small—spec'd PRs are 10% more likely to need rework, and the practical difference is about one extra rework event per 83 PRs—but it is in the wrong direction. Specifications are associated with more rework, not less.

[a] Odds ratio: spec'd PRs are 1.18× as likely to be reworked as unspec'd PRs.

5.3 H3: Specification Quality Reduces Defects

H1 and H2 tested specification *presence*—whether having any specification artifact matters. H3 and H4 test specification *quality*—whether, among PRs that already have specifications, better-written specifications predict better outcomes. This is the stronger version of the SDD claim: not just "write a spec" but "write a *good* spec." We consider H3 and H4 *exploratory* rather than confirmatory: the quality measure has limited construct validity (human–LLM agreement $\rho = 0.42$ on $N = 38$).

Quality is the mean of seven rubric dimensions (described in Section 4.2), each scored 0–100. These dimensions were chosen to align with the quality criteria that SDD tools themselves prescribe (Section 4.2). The scoring is automated with limited human validation ($\rho = 0.42$ on 38 PRs; Section 4.2)—a limitation we acknowledge (Section 7). The quality score measures *formal* specification completeness (are error states described? are acceptance criteria present?) rather than *functional* correctness (are the *right* error states described? are the acceptance criteria valid for this domain?). A specification can score highly on every dimension and still specify the wrong behavior.

[2] Odds ratio = 1.13; Cohen's $d = 0.07$, well below the conventional "small effect" threshold of 0.20.

Table 5 H3: Specification quality and defect-introduction rates.

Method	Coefficient	p-value	Interpretation
Controlled (size only†)	−0.002	0.080	Approaches significance
Within-author	−0.0003 [−0.0007, +0.0001]	0.164	Not significant

Table 6 H4: Specification quality and rework rates.

Method	Coefficient	p-value	Interpretation
Controlled (size + repo)	−0.000	0.920	Quality does not predict rework
Within-author	+0.000 [−0.000, +0.000]	0.860	Effectively zero

For PRs whose specification source is a linked GitHub issue, we fetched the issue body and concatenated it with the PR description before scoring, giving the LLM scorer access to the actual pre-implementation specification rather than just the PR summary. 25,209 PRs (28.6% of total) received quality scores across all 119 repositories; 17,973 of these were enriched with linked issue content.

Of the 22,672 quality-scored PRs in SZZ-covered repos, 19,688 remain after restricting to authors with ≥2 PRs and dropping rows with missing controls.

† Repository adjustments caused the model to fail; this estimate adjusts for PR size only.

Identified from 2,030 authors with treatment variation.

H3 is not supported. The controlled estimate ($p = 0.080$) approaches significance but the effect is tiny: a 10-point quality improvement predicts a 2 percentage-point defect reduction. The within-author estimate is smaller and not significant ($p = 0.164$): a 10-point improvement predicts a 0.3pp reduction. The quality signal does not hold up under stronger controls and is non-significant across all robustness checks (Section 5.6).

5.4 H4: Specification Quality Reduces Rework

Identified from 2,297 authors with treatment variation (same scored subsample as H3).

H4 is not supported. Specification quality has no meaningful relationship with rework rates within-author. The coefficient is effectively zero ($p = 0.860$). The controlled logistic regression is also non-significant ($p = 0.920$).

5.5 H5: Specifications Constrain AI Scope

Analysis uses log code churn (additions + deletions) as outcome. PRs with zero churn are excluded (these represent missing data rather than true zero-change events). AI detection combines regex pattern matching with structured AI disclosure parsing (Section 4.2), identifying 5,989 AI-tagged PRs—2.3× more than co-author tags alone.

AI analysis identified from 224 authors with treatment variation; interaction from 366.

H5 is not supported. Rather than constraining scope, specifications accompany *larger* PRs for both AI-tagged (+12.0%) and human-authored (+14.0%) work. The interaction term is effectively zero ($p = 0.997$): specifications have the same relationship with code churn for AI and human PRs alike—the same confounding-by-indication pattern seen in H1 and H2. Harder tasks get specs and produce more code.

Table 7 H5: Specifications and AI-generated code scope.

Group	Within-author coef	p-value	Interpretation
AI-tagged PRs	+0.120	0.030	Spec'd AI PRs are *larger*
Human PRs	+0.140	< 0.001	Spec'd human PRs are *larger*
Interaction (spec × AI)	+0.000 [−0.101, +0.102]	0.997	No differential scope constraint

5.6 Robustness Checks

Twelve additional analyses test whether the null result is an artifact of our primary measures or insufficient controls.

5.6.1 Alternative Outcome Measures

We tested two additional outcomes beyond SZZ-traced defects and rework: *escaped defects* (PRs whose CI passed but were subsequently followed by a fix or revert) and *strict escaped* (escaped PRs where the follow-up explicitly indicates a fix or regression). Both are directionally positive (specs accompany more escapes, not fewer) but not significant (escaped: $p = 0.333$; strict escaped: $p = 0.108$; within-author LPM, $N = 81{,}647$). No outcome measure shows specifications reducing defects at $p < 0.05$.

5.6.2 Incremental Validity Beyond JIT Features

We tested whether specification information adds predictive power beyond the 14 JIT defect prediction features [13] on 61,192 PRs with complete JIT features and SZZ outcomes. Spec presence is statistically significant when added to the JIT model ($p = 0.003$) but adds negligible predictive power ($\Delta R^2 = 0.0002$; AIC decrease of 7 points). Spec quality adds nothing ($p = 0.142$). JIT features account for nearly all the predictive power; knowing whether a PR has a specification tells you almost nothing beyond what the code-change metrics already reveal.

5.6.3 Quality Dimensions: Individual and Validated Subsets

We tested each of the seven quality dimensions independently against SZZ bugs and rework (within-author LPM, 19,688 PRs). No individual dimension is significant for defects at $p < 0.05$; every coefficient rounds to zero. Scope boundaries ($p = 0.053$) and behavioral specificity ($p = 0.083$) are closest, but neither survives Bonferroni correction for seven simultaneous tests ($p < 0.007$ required). No dimension shows any relationship with rework.

To test whether measurement noise in weakly validated dimensions attenuates the quality signal, we compared composites restricted to the four dimensions with significant human–LLM agreement ($\rho = 0.39$–0.45: behavioral specificity, scope boundaries, dependency context, outcome clarity) against composites using the three unvalidated dimensions ($\rho = 0.17$–0.22: acceptance criteria, error states, data contracts).

All three composites are non-significant for defects. The validated and unvalidated composites perform similarly, consistent with the quality score capturing noise rather than a real defect-predictive signal. For rework, all three composites produce coefficients indistinguishable from zero.

Table 8 Validated vs. unvalidated quality dimension composites.

Quality composite	→ bugs coef	p	→ rework coef	p
All 7 dimensions	−0.0003	0.164	+0.0000	0.860
4 validated dimensions only	−0.0002	0.184	+0.0000	0.913
3 unvalidated dimensions only	−0.0003	0.158	+0.0001	0.663

Table 9 Subgroup analysis: specification effects across populations.

Subgroup	→ bugs coef	p	→ rework coef	p
All non-bot PRs (88,052)	+0.014	0.003	+0.012	0.001
Human-only, no AI (82,063)	+0.015	0.003	+0.011	0.004
AI-tagged, non-bot (5,989)	+0.011	0.424	+0.010	0.399
Zero-AI repos (3,562)	−0.016	0.377	−0.033	0.014
Low-AI repos (51,877)	+0.019	0.001	+0.009	0.061
High-AI repos (32,613)	+0.010	0.185	+0.018	0.003

5.6.4 Repo-Level Analysis

Aggregating to the repository level ($N = 98$ repos with $\geq$50 PRs), specification rate has no relationship with either defect rate (Spearman $\rho = -0.11$, $p = 0.27$) or rework rate ($\rho = -0.05$, $p = 0.64$). Repos that write more specs do not have fewer bugs or less rework.

5.6.5 Subgroup Analysis

SDD tools are marketed primarily for AI-assisted workflows. We test whether the null result holds across human-only PRs, AI-tagged PRs, and repos stratified by AI adoption rate. As in all analyses, bot PRs are excluded.

One subgroup breaks the pattern. In repositories with no AI-tagged PRs, specifications are associated with *less* rework (−3.3pp, $p = 0.014$). This is the only result in the dataset consistent with the SDD rework claim, and it appears precisely where AI involvement is absent—in purely human workflows. The finding comes from a small subsample (3,562 PRs), is one of six subgroup tests, and does not remain significant after correcting for running six tests simultaneously. We note it as suggestive but not conclusive. That the protective effect appears only where AI is absent, not where AI is present, is itself inconsistent with the SDD vendor claim that specifications are *most* valuable for AI-assisted work.

For all other subgroups, the pattern holds. For human-authored PRs, specs are associated with *more* defects and *more* rework—the confounding-by-indication pattern. For AI-tagged PRs, the coefficients are positive but not significant ($p = 0.42$ for defects, $p = 0.40$ for rework). The null holds whether a repo has low or high AI adoption.

5.6.6 High-Quality Specs Only

A likely reviewer objection is that our `specd` measure is too loose—that linked Jira tickets are not "real" specifications in the SDD sense, and only high-quality structured specs should show the benefit. We test this directly by restricting to top-quartile (quality score ≥ 58) and top-decile (≥ 66) specs. Quality thresholds are computed from the scored subset's distribution (median = 48, p75 = 58, p90 = 66).

Table 10 High-quality specification thresholds vs. defect rates.

Test	→ bugs coef	p	Identifying authors
Top-quartile specs vs. all others	−0.007	0.190	1,370
Top-decile specs vs. all others	−0.005	0.545	751
High-quality vs. NO spec (low-quality excluded)	−0.006	0.348	1,117
AI-tagged + high-quality spec	−0.005	0.790	175

Table 11 Three-tier dose-response. PRs are split into: high-quality specs (top-quartile quality score ≥ 58), all other spec'd PRs (below top-quartile or unscored), and PRs with no specification artifact. If specification quality matters, defect and rework rates should decrease from bottom to top. Coefficients from within-author analysis; "no spec" is the reference group. Bug analysis restricted to 103 SZZ-covered repos.

Tier	Raw bug rate	→ bugs coef	p	→ rework coef / p
High-quality spec ($N = 5{,}253$)	12.6%	−0.004	0.566	+0.017 / $p = 0.006$
Any other spec ($N = 16{,}505$)	14.5%	+0.020	< 0.001	+0.010 / $p = 0.012$
No spec ($N = 56{,}056$)	12.0%	—	ref.	— / ref.

No quality threshold produces a significant defect reduction. The top-quartile result (−0.7pp, $p = 0.190$) and top-decile result (−0.5pp, $p = 0.545$) are both non-significant—the opposite of a dose-response gradient. The AI + high-quality spec result ($p = 0.790$), the closest test to the agentic SDD workflow vendors are selling, shows no signal.

We also test a three-tier dose-response by splitting PRs into three mutually exclusive groups: high-quality specs (top-quartile scored PRs), any other spec (spec'd but below top-quartile or unscored), and no spec at all. If specification quality matters, the tiers should show a monotonic gradient.

The expected gradient does not appear. High-quality specs show a raw defect rate (12.6%) indistinguishable from no spec (12.0%), while ordinary specs show the highest rate (14.5%). Within-author, high-quality specs produce a non-significant −0.4pp reduction ($p = 0.566$); ordinary specs produce a significant +2.0pp *increase* ($p < 0.001$). Neither tier reduces rework. The pattern is consistent with confounding, not a quality gradient.

5.6.7 JIT Features as Primary Controls

A reviewer concern is that size controls (log additions, deletions, files) are insufficient—the JIT defect prediction framework [13] captures task complexity dimensions (entropy, file age, developer experience) that may confound the specification–defect association. We re-run H1 and H2 with 13 JIT features as additional controls alongside size controls (subsystem experience excluded for zero variance).

For H1, the spec coefficient drops 55% and loses significance when JIT features are added. For H2, the coefficient drops 23% but remains significant ($p = 0.024$). A discrepancy with PSM (below), which finds $p = 0.146$ for rework, likely reflects functional form differences: JIT regression assumes linearity while PSM does not. Either way, the rework association is directionally *positive* under both methods—the direction never supports the SDD claim.

Table 12 JIT features as primary controls.

Test	Size controls only	+ JIT controls	Change
H1: specs → bugs (coef)	+0.014 ($p = 0.003$)	+0.006 ($p = 0.229$)	−55%
H2: specs → rework (coef)	+0.012 ($p = 0.001$)	+0.009 ($p = 0.024$)	−23%

Table 13 Propensity score matching results.

Outcome	Spec'd (matched)	Unspec'd (matched)	Difference
Defect rate (17,375 pairs)	14.7%	15.2%	−0.6pp ($p = 0.133$)
Rework rate (19,648 pairs)	15.2%	14.7%	+0.5pp ($p = 0.146$)

5.6.8 Propensity Score Matching

We pair each spec'd PR with an unspec'd PR having a similar JIT complexity profile[3], testing whether confounding by indication explains the association.

All 16 covariates are well-balanced after matching. For defects, the matched rates are indistinguishable—PSM eliminates the raw association entirely. For rework, the difference is +0.5pp and not significant ($p = 0.146$). Both associations are explained by the JIT risk profile of tasks that receive specifications.

5.6.9 Temporal Analysis: The SDD Era

We restrict to the most recent three months (January–March 2026), when specification rates are highest (29.3%) and AI adoption most prevalent (8.8%), to test whether the null result is a historical artifact.

In the period of highest SDD adoption, specifications are associated with more rework (+1.6pp, $p = 0.001$) and directionally more defects (+0.4pp, $p = 0.435$, not significant). For AI-tagged PRs, specs have no effect in either direction. The null result persists in the data most representative of the SDD era. SZZ-traced defect rates for recent months are likely underestimates due to right-censoring, but this affects absolute rates, not the relative spec'd vs. unspec'd comparison.

5.6.10 Complexity Stratification

We stratify by the top 20% of three complexity measures (code churn, change entropy, composite JIT risk) to test whether specifications help most on the hardest tasks.

The result is opposite to the SDD prediction: on the hardest tasks, specifications are associated with *more* defects and *more* rework—confounding by indication is strongest where specs should help most. On simpler tasks, the association disappears. Specification quality (H3/H4) is null across all strata.

[3] Nearest-neighbor matching on propensity scores (predicted probability of being spec'd based on JIT features), caliper = 0.05 SD.

Table 14 Temporal analysis: recent three months vs. full dataset.

Test	Recent 3 months coef	p	Full dataset coef	p
Specs → SZZ bugs	+0.004	0.435	+0.014	0.003
Specs → rework	+0.016	0.001	+0.012	0.001
AI + specs → bugs	+0.003	0.822	—	—
AI + specs → rework	+0.016	0.212	—	—
Spec × AI interaction (H5)	+0.011	0.861	+0.000	0.997

Table 15 Specification effects stratified by task complexity (within-author LPM).

Stratum	→ bugs coef	p	→ rework coef	p
Top 20% churn	+0.040	< 0.001	+0.051	< 0.001
Bottom 80% churn	+0.007	0.114	+0.005	0.238
Top 20% entropy	+0.024	0.057	+0.039	< 0.001
Bottom 80% entropy	+0.000	0.986	+0.005	0.201
Top 20% JIT risk	+0.028	0.028	+0.039	< 0.001
Bottom 80% JIT risk	+0.002	0.713	+0.008	0.059

5.6.11 Issue-Linked Specifications

A further objection is that our `specd` measure includes weak signals (Jira ticket IDs, template sections). We restrict to the 18,600 PRs that link to a GitHub issue—the closest proxy for a pre-implementation specification. For issue-linked PRs with quality scores, we also fetched the linked issue body and scored the combined content, giving the quality rubric access to the actual specification rather than just the PR summary.

Issue-linked specifications show the same pattern: defect associations are positive or null, never protective. Even top-20% quality GitHub issues show no defect reduction ($p = 0.363$). For AI-tagged PRs with issue-linked specs, both defects ($p = 0.436$) and rework ($p = 0.192$) are non-significant.

5.6.12 Summary

The null result holds across four outcome measures, JIT controls, PSM, seven quality dimensions, PR-level and repo-level analysis, six subgroup cuts, five complexity strata, issue-linked specs, multiple quality thresholds, dose-response tests, and temporal restriction. The one exception is zero-AI repositories (−3.3pp rework, $p = 0.014$). No specification measure robustly predicts fewer defects under any operationalization.

6 Discussion

6.1 Confounding by Indication

The pattern across all five hypotheses is identical: unadjusted analysis suggests specifications accompany worse outcomes; within-author analysis attenuates but does not eliminate the reversed association; propensity score matching on JIT features eliminates it entirely, confirming confounding by indication [20]. Developers rationally invest specification effort in proportion to task complexity.

Table 16 Issue-linked specifications vs. other operationalizations (within-author LPM).

Test	→ bugs coef	p	→ rework coef	p
Issue-linked vs no spec	+0.011	0.037	+0.014	< 0.001
AI + issue vs AI + no spec	+0.014	0.436	+0.017	0.192
Human + issue vs human + no spec	+0.012	0.035	+0.013	0.002
Top-20% quality issue vs no spec	−0.007	0.363	+0.019	0.007
Issue-linked vs ticket-only	+0.007	0.637	+0.026	0.025

JIT risk profiles confirm this: spec'd PRs have more lines added (1.8×), touch older files (2.3×), and involve more prior developers (1.3×). Notably, spec'd PR authors have *less* experience (median 64 vs. 116 prior commits), consistent with specifications being a support mechanism for harder tasks.

The within-author null is consistent with two readings: specifications genuinely have no effect, or a small positive effect is concealed by residual task selection. We cannot distinguish these with observational data. What we can say is that the estimates are *directionally reversed* across most tests, and PSM eliminates both associations entirely. Any protective effect is smaller than our study can detect.

6.2 Why Specifications May Not Prevent Defects

For a specification to prevent a defect, every link in a causal chain must hold:

1. The developer must correctly identify the problem.
2. The developer must know in advance what will solve the problem.
3. The developer must know exactly how to execute the solution.
4. The developer must be able to perfectly describe the solution in natural language.
5. The natural language description must not be misinterpreted by the AI.
6. The AI must not introduce bugs in its implementation.
7. The specification must not have substantial omissions—the developer must specify what they do not yet know they do not know.

Each condition is individually uncertain; together the probability of the full chain succeeding diminishes rapidly. Defects arise precisely where human understanding is incomplete—and specifications are written by the same humans whose incomplete understanding produces the defects.

SDD vendors conflate *directing* the AI (telling it what to build) with *ensuring quality* (fewer defects, less rework). Specifications may be effective at direction, but an agent faithfully executing a spec will reproduce every gap, ambiguity, and wrong assumption in that spec. Direction is not quality.

6.3 Vendor Evidence

No SDD tool vendor has published empirical evidence for these quality claims. GitHub describes Spec Kit as "an experiment" [8] with no effectiveness data. Third-party evaluations are negative: Böckeler found the overhead "overkill" [6]; Eberhardt found it roughly ten times slower [9]. None of the 119 repositories show evidence of SDD tooling (zero matches for `.specify/`, `.speckit/`, `.kiro/`, `spec-kit` across 100,247 PRs). Vendors have the data

to conduct controlled evaluations; the absence of published results alongside specific quality claims is a gap that experiments could close.

We do not argue that thinking about requirements is wasted effort. We argue that the *products* claiming to reduce defects through specification artifacts have not demonstrated that they do.

7 Threats to Validity

7.1 Construct Validity

Three gaps separate what we measure from what SDD vendors claim.

We detect specification artifacts, not specification processes. If the benefit comes from the *thinking* rather than the *artifact*, our measure is too coarse. However, the within-author design controls for consistent habits, and H3/H4 test quality *within* spec'd PRs—if deeper thinking produced better artifacts, those artifacts should predict better outcomes. They do not.

We test organic specifications, not SDD tool output. Spec Kit generates structured documents through a guided workflow. A linked GitHub issue is not the same thing. The high-quality spec subsample (top-quartile, scored on SDD-prescribed dimensions) is the closest approximation, and it too shows no robust effect.

We cannot observe agentic workflows directly. Even for AI-tagged PRs with specifications, we do not know whether the spec was used as agent input. The 175-author AI + high-quality spec subsample is the closest proxy we have.

7.2 Internal Validity

Defect detection is noisy. Basic SZZ misattributes 46–71% of bug-introducing changes [7]. The noise may be differential: spec'd PRs have 1.8× more lines added and touch older files, creating more `git blame` targets. Two observations partially mitigate this: the rework measure (H2) does not depend on SZZ and shows the same pattern, and propensity score matching eliminates the defect association entirely. Nonetheless, confounding by indication and differential SZZ noise are difficult to disentangle.

Quality scoring is automated with limited validation. Our LLM rubric measures *formal* quality (are error states described?) not *functional* quality (are the *right* error states described?). The scoring was applied only to PRs with substantial description text, creating a non-random subsample.

AI detection relies on voluntary tagging. The false-negative rate is unknown. If many untagged PRs involved AI, the H5 comparison would be diluted toward the null.

The control group may be contaminated. Some "unspec'd" PRs may have had specifications shared verbally or in external tools, attenuating any treatment effect.

Confounding by indication is the primary threat. The within-author design compares each developer to themselves, but within-author task selection remains possible. The JIT-controlled models and propensity score matching address this further and both confirm the null for defects.

Time-varying confounding. An author's skill and tooling may change over the observation period. The within-author estimator assumes these are stable.

7.3 External Validity

Open-source only. All 119 repositories are open-source. SDD tools target commercial teams where specification practices and defect economics may differ. **Convenience sample.** Repositories were not randomly selected; results cannot be generalized beyond the sample. **Pre-agentic era.** Most PRs predate widespread agentic AI coding tools. Within the 5,989 AI-tagged PRs, specs show no effect on defects ($p = 0.424$) or rework ($p = 0.399$), but we cannot rule out that true agentic workflows would produce different results.

7.4 Conclusion Validity

Multiple comparisons. We test five hypotheses without correction; approximately 0.25 false positives are expected by chance. **Power for AI interaction.** The H5 interaction is identified from 366 authors; the null ($p = 0.997$) is not a power issue—the coefficient is effectively zero. **No pre-registration.** All analyses are post-hoc. The twelve robustness checks show stability across specifications, but a pre-registered confirmatory study would provide stronger evidence.

8 Conclusion

We tested five SDD vendor claims against 88,052 pull requests across 119 repositories, scoring specification artifacts on the same quality dimensions SDD tools prescribe and comparing each developer's spec'd work to their own unspec'd work. None of the five hypotheses are supported. Specification quality does not predict fewer defects ($p = 0.164$) or less rework ($p = 0.860$). Specifications do not constrain AI-generated code scope ($p = 0.997$). Across twelve robustness checks, the null holds. Specification presence proxies for task complexity, not quality improvement.

One exception: in repositories with zero AI adoption, specifications are associated with less rework (-3.3pp, $p = 0.014$)—the only protective signal, appearing where AI is absent.

Three caveats bound these findings: we test organic specification artifacts, not SDD tool output (substantial construct gap); SZZ has 46–71% misattribution noise; and our open-source convenience sample may not generalize. Whether purpose-built SDD tooling would produce different results is a question vendors can answer with the data they already have.

Specifications may still have value—as auditable records of intent, as task-batching mechanisms for AI agents, and as accountability artifacts that invite review scrutiny. These are real benefits. They are not "fewer defects," and the evidence does not support the claim that specifications are most valuable when AI implements the code.

References

1. GitHub Copilot Academy. Spec-Driven Development, 2025. URL: `https://copilot-academy.github.io/workshops/immersive-experience/spec_driven_development`.
2. Joshua D. Angrist and Jörn-Steffen Pischke. *Mostly Harmless Econometrics: An Empiricist's Companion*. Princeton University Press, 2009.
3. Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, and Dave Thomas. Manifesto for Agile Software Development, 2001. URL: `https://agilemanifesto.org`.
4. Barry W. Boehm. *Software Engineering Economics*. Prentice Hall, 1981.
5. Marc Brooker. Kiro and the Future of AI Spec-Driven Software Development, 2025. URL: `https://kiro.dev/blog/kiro-and-the-future-of-software-development/`.

6 Birgitta Böckeler. Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl, 2025. URL: https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html.

7 Daniel Alencar da Costa, Shane McIntosh, Weiyi Shang, Uirá Kuber, Roberta Coelho, and Ahmed E. Hassan. A Framework for Evaluating the Results of the SZZ Approach for Identifying Bug-Introducing Changes. *IEEE Transactions on Software Engineering*, 43(7):641–657, 2017.

8 Den Delimarsky. Spec-Driven Development with AI: Get Started with a New Open Source Toolkit, 2025. URL: https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/.

9 Colin Eberhardt. Putting Spec Kit Through Its Paces: Radical Idea or Reinvented Waterfall?, 2025. URL: https://blog.scottlogic.com/2025/11/26/putting-spec-kit-through-its-paces-radical-idea-or-reinvented-waterfall.html.

10 Henning Femmer, Daniel Méndez Fernández, Stefan Wagner, and Sebastian Eder. Rapid Quality Assurance with Requirements Smells. *Journal of Systems and Software*, 123:190–213, 2017.

11 Leigh Griffin and Ray Carroll. Spec Driven Development: When Architecture Becomes Executable, 2026. URL: https://www.infoq.com/articles/spec-driven-development/.

12 Hua Guo, Yunhong Ji, and Xuan Zhou. Sedeve-Kit, a Specification-Driven Development Framework for Building Distributed Systems. *arXiv preprint*, 2025. URL: https://arxiv.org/abs/2509.11566.

13 Yasutaka Kamei, Emad Shihab, Bram Adams, Ahmed E. Hassan, Audris Mockus, Anand Sinha, and Naoyasu Ubayashi. A Large-Scale Empirical Study of Just-in-Time Quality Assurance. *IEEE Transactions on Software Engineering*, 39(6):757–773, 2013.

14 Srinivas Rao Marri. Constitutional Spec-Driven Development: Enforcing Security by Construction in AI-Assisted Code Generation. *arXiv preprint*, 2026. URL: https://arxiv.org/abs/2602.02584.

15 Jürgen Mund, Henning Femmer, Daniel Méndez Fernández, and Jonas Eckhardt. Does Quality of Requirements Specifications Matter? Combined Results of Two Empirical Studies. In *Proceedings of the 9th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–10, 2015.

16 Rich Naszcyniec. How Spec-Driven Development Improves AI Coding Quality, 2025. URL: https://developers.redhat.com/articles/2025/10/22/how-spec-driven-development-improves-ai-coding-quality.

17 Jerzy Neyman and Elizabeth L. Scott. Consistent Estimates Based on Partially Consistent Observations. *Econometrica*, 16(1):1–32, 1948.

18 Deepak Babu Piskala. Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants. *arXiv preprint*, 2026. URL: https://arxiv.org/abs/2602.00180.

19 Yannick Roy. The Kitchen Loop: User-Spec-Driven Development for a Self-Evolving Codebase. *arXiv preprint*, 2026. URL: https://arxiv.org/abs/2603.25697.

20 Maribel Salas, Albert Hofman, and Bruno H. Ch. Stricker. Confounding by Indication: An Example of Variation in the Use of Epidemiologic Terminology. *American Journal of Epidemiology*, 149(11):981–983, 1999.

21 Forrest Shull, Victor Basili, Barry Boehm, A. Winsor Brown, Patricia Costa, Mikael Lindvall, Dan Port, Ioana Rus, Roseanne Tesoriero, and Marvin Zelkowitz. What We Have Learned About Fighting Defects. In *Proceedings of the 8th International Symposium on Software Metrics*, pages 249–258, 2002.

22 Ryan Yanchuleff. From Chat to Specs: A Deep Dive into AI-Assisted Development with Kiro, 2025. URL: https://kiro.dev/blog/from-chat-to-specs-deep-dive/.

23 François Zaninotto. Spec-Driven Development: The Waterfall Strikes Back, 2025. URL: https://marmelab.com/blog/2025/11/12/spec-driven-development-waterfall-strikes-back.html.

24 Christo Zietsman. The Specification as Quality Gate: Three Hypotheses on AI-Assisted Code Review. *arXiv preprint*, 2026. URL: https://arxiv.org/abs/2603.25773.

25 Jacek Iwerski, Thomas Zimmermann, and Andreas Zeller. When Do Changes Induce Fixes? In *Proceedings of the 2nd International Workshop on Mining Software Repositories (MSR)*, pages 1–5, 2005.